إجابات سريعة: الفهارس

المدة
١٦ دقيقة

المواد
هاتفك وعليه درسي، لخطوات Python، صديق، للعبة «خمّن رقمي»

أهداف التعلّم

- تشرح لماذا تصير قراءة كل صف بطيئة.
- تعدّ نظرات البحث الخطّي والثنائي في Python.
- تقرّر أيّ الأعمدة تستحق فهرسًا.

النشاط: خمّن رقمي

١. الأزواج (دقيقتان): يختار أحد الزميلين سرًّا رقمًا من ١ إلى ١٠٠.
٢. التخمين (٥ دقائق): خمّن أولًا ١، ٢، ٣... بالترتيب، وعُدّ التخمينات.
٣. التنصيف (٥ دقائق): خمّن الآن دائمًا وسط ما بقي، والإجابة «أعلى» أو «أدنى». وعُدّ من جديد.
٤. المقارنة (٤ دقائق): قارنوا الأعداد. لماذا لا يحتاج التنصيف أبدًا إلى أكثر من ٧؟

دع التلاميذ يسجّلون النتائج على الورق، فسجلّ التخمينات يُظهر النمط.

للنقاش

- لماذا يصير البحث الخطّي أبطأ بعشر مرات على صفوف أكثر بعشر مرات؟
- لماذا يحتاج التنصيف إلى بيانات مرتّبة؟
- ماذا يكلّف الفهرس؟

تحقّق من نفسك

- اكتب كم نظرة يحتاج إليها التنصيف لـ ١٠٠٠ صف، ولماذا.

جرّبها هذا الأسبوع

العب «خمّن رقمي» من ١ إلى ١٠٠ مع صديق. وخمّن الأوسط دائمًا. هل تستطيع الفوز دائمًا في سبعة تخمينات؟ ولماذا سبعة؟

ورقة عمل

إجابات سريعة: الفهارس

هندسة البيانات، المستوى ٣، الدرس ٤

الاسم _____ الفصل _____ التاريخ _____

١ لماذا لا نضع فهرسًا على كل عمود؟

- ◯ لا تسمح SQL إلا بفهرس واحد
- ◯ كل فهرس يأخذ مساحة ويُبطئ كل صف جديد
- ◯ الفهارس تجعل الإجابات خاطئة

٢ البحث الخطّي. يجب أن تعيد linear(ids, target) عدد النظرات التي احتاجت إليها لإيجاد target (أو للاستسلام). فلـ [5, 3, 9, 1] و9 يكون ذلك ٣ نظرات. أصلح خطأ الواحد الزائد أو الناقص.

```
def linear(ids, target):
    looks = 0
    for x in ids:
        if x == target:
            return looks
        looks = looks + 1
    return looks
```

اكتب برنامجك:

٣ كم مرة تستطيع تنصيف المليون حتى يبقى واحد؟ ماذا يطبع هذا البرنامج؟

```
n = 1000000
looks = 0
while n > 1:
    n = n // 2
    looks = looks + 1
print(looks)
```

- ◯ 500000
- ◯ 1000
- ◯ 19

٤ البحث الثنائي في قائمة مرتّبة. يجب أن تنصّف binary(ids, target) البحث في كل نظرة وتعيد عدد النظرات. لكن مقارنة واحدة ترسله في الاتجاه الخاطئ: جدها وأصلحها. ومن ١ إلى ١٠٠ يجب ألّا يحتاج أبدًا إلى أكثر من ٧ نظرات.

```
def binary(ids, target):
    lo, hi = 0, len(ids) - 1
    looks = 0
    while lo <= hi:
        mid = (lo + hi) // 2
        looks = looks + 1
        if ids[mid] == target:
            return looks
        elif ids[mid] > target:
            lo = mid + 1
        else:
            hi = mid - 1
    return looks
```

اكتب برنامجك:

٥ يُبحث في جدول إعارات مكتبة مدرسية حسب العضو وحسب الكتاب كل يوم. أيّ الأعمدة يستحق فهرسًا؟ اختر كل ما ينطبق.

- ☐ لون غلاف الكتاب
- ☐ member_id
- ☐ notes، الذي لا يبحث فيه أحد
- ☐ book_id

✎ أين تنصّف البحث في حياتك الحقيقية، وأين تقرأ واحدًا بعد آخر؟

تحقّق من إجاباتك

إجابات سريعة: الفهارس

١ **كل فهرس يأخذ مساحة ويُبطئ كل صف جديد**

تبادل الفهارس المساحة والكتابة الأبطأ بالبحث الأسرع: فكل صف جديد يجب أن يُضاف إلى كل فهرس.

٢

```
def linear(ids, target):
    looks = 0
    for x in ids:
        looks = looks + 1
        if x == target:
            return looks
    return looks
```

كل عنصر تفحصه نظرة واحدة، بما فيه العنصر الذي تجده، فعُدّ قبل المقارنة.

٣ **19**

تنصيف المليون مرة بعد مرة يصل إلى ١ بعد ١٩ خطوة، ومع الفحص الأخير يكون ذلك نحو ٢٠ نظرة.

٤

```
def binary(ids, target):
    lo, hi = 0, len(ids) - 1
    looks = 0
    while lo <= hi:
        mid = (lo + hi) // 2
        looks = looks + 1
        if ids[mid] == target:
            return looks
        elif ids[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return looks
```

يحدّد lo وhig الجزء الباقي. وحين يكون الأوسط أصغر من المطلوب، فالمطلوب فوقه، فيتجاوز lo قيمة mid، وإلا تحرّك hi.

٥ **book_id ،member_id**

ضع الفهرس على ما تبحث به أو تربط به أو ترتّب به كثيرًا.